CEFET-MG | DEPARTAMENTO DE COMPUTAÇÃO

Computação Gráfica

Interpolação de Textura

Prof. Dr. Raulivan Rodrigo

Contextualização

Chegamos ao ponto em que unimos tudo o que estudados até aqui:

- Álgebra Linear (Matrizes)
- Pipeline Programável (Shaders)
- Gestão de Memória da GPU (VAO/VBO) e, agora,
- Leitura de Arquivos de Disco e Filtros de Textura.

Contextualização

- Para ler um arquivo de imagem (JPG/PNG) e extrair os bytes
- É necessário as biblioteca Pillow
- pip install glfw PyOpenGL numpy Pillow

Interpolação de Textura

Na prática

cubo_textura.py

Código: https://github.com/raulivan/computacao-grafica-python/blob/main/AULA17/cubo_textura.py

cubo_textura.py U

AULA17 > cubo_textura.py > ...

```
1  import glfw
2  from OpenGL.GL import *
3  import OpenGL.GL.shaders
4  import numpy as np
5  from PIL import Image
6
```

Interpolação de Textura

```
7   # HADERS com suporte a textura
8   VERTEX_SHADER = """
9   #version 330 core
10  layout (location = 0) in vec3 aPos;     // Porta 0: Geometria (X, Y, Z)
11  layout (location = 1) in vec2 aTexCord; // Porta 1: Mapeamento (U, V)
12
13  out vec2 TexCord; // Passa o UV para o Fragment Shader
14  uniform mat4 MVP; // Matriz Mestre
15
16  void main() {
17      gl_Position = MVP * vec4(aPos, 1.0);
18      TexCord = aTexCord;
19  }
20  """
```

Interpolação de Textura

```
22  FRAGMENT_SHADER = """
23  #version 330 core
24  in vec2 TexCord;
25  out vec4 FragColor;
26
27  // Slot de memória da GPU onde a imagem estará vinculada
28  uniform sampler2D textura_bloco;
29
30  void main() {
31      // Lê a cor exata do pixel da textura naquela coordenada UV
32      FragColor = texture(textura_bloco, TexCord);
33  }
34  """
```

```python
# Ler a imagem do HD
def carregar_textura_do_disco(caminho_arquivo):
    print(f"Carregando: {caminho_arquivo}...")

    try:
        # abre o arquivo e o joga na memória RAM da CPU
        imagem = Image.open(caminho_arquivo)

        # ATENÇÃO AQUI: O OpenGL considera a coordenada (0,0) da imagem
        # no canto INFERIOR esquerdo. As imagens de computador padrão usam (0,0) no
        # canto SUPERIOR esquerdo.
        # Precisamos inverter a imagem verticalmente!
        imagem = imagem.transpose(Image.FLIP_TOP_BOTTOM)

        # Converte para RGBA para garantir que teremos 4 canais (Red, Green, Blue, Alpha)
        imagem_dados = imagem.convert("RGBA").tobytes()
        largura, altura = imagem.size
        print(f" deu certim: {largura}x{altura}")

    except FileNotFoundError:
        print(" ixi, deu ruim. Gerando textura de 'Erro' (Rosa/Preto)...")
        largura, altura = 2, 2
        imagem_dados = np.array([
            255, 0, 255, 255,   0, 0, 0, 255, # Linha inferior (Rosa, Preto)
            0, 0, 0, 255,       255, 0, 255, 255  # Linha superior (Preto, Rosa)
        ], dtype=np.uint8).tobytes()

```

Interpolação de Textura

```
63      # RAM -> VRAM: 1 ID de Textura para a GPU
64      textura_id = glGenTextures(1)
65
66      # liga a textura
67      glBindTexture(GL_TEXTURE_2D, textura_id)
68
69      # comfigura para alto realismo (Filtro Bilinear/Trilinear)
70      # O filtro MIN atua quando a imagem está longe.
71      # O filtro MAG atua quando a câmera chega muito perto.
72      glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER, GL_LINEAR_MIPMAP_LINEAR)
73      glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER, GL_LINEAR)
74
75      # Configuração de Repetição
76      # Se o UV passar de 1.0, a imagem se repete como papel de parede
77      glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_S, GL_REPEAT)
78      glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_T, GL_REPEAT)
79
```

Interpolação de Textura

Na prática

```
80      # envia os bytes da RAM (CPU) para a VRAM (Placa de Vídeo)
81      glTexImage2D(GL_TEXTURE_2D, 0, GL_RGBA, largura, altura,
82                   0, GL_RGBA, GL_UNSIGNED_BYTE, imagem_dados)
83
84      # geração de Mipmaps
85      # isso que evita o "serrilhado piscante" quando a textura está muito longe ou inclinada.
86      glGenerateMipmap(GL_TEXTURE_2D)
87
88      #fecha a textura
89      glBindTexture(GL_TEXTURE_2D, 0)
90
91      return textura_id
92      # fim do método carregar_textura_do_disco
```

Interpolação de Textura

```
95  def matriz_perspectiva(fov, aspecto, perto, longe):
96      f = 1.0 / np.tan(np.radians(fov) / 2.0)
97      m = np.zeros((4, 4), dtype=np.float32)
98      m[0,0] = f / aspecto; m[1,1] = f; m[2,2] = (longe + perto) / (perto - longe)
99      m[2,3] = (2.0 * longe * perto) / (perto - longe); m[3,2] = -1.0
100     return m
101
102 def matriz_translacao(x, y, z):
103     return np.array(
104         [[1,0,0,x],
105          [0,1,0,y],
106          [0,0,1,z],
107          [0,0,0,1]
108          ], dtype=np.float32)
109
```

```
110    def matriz_rotacao_x(g):
111        c,s = np.cos(np.radians(g)), np.sin(np.radians(g));
112        return np.array([
113            [1,0,0,0],
114            [0,c,-s,0],
115            [0,s,c,0],
116            [0,0,0,1]
117            ], dtype=np.float32)
118
119    def matriz_rotacao_y(g):
120        c,s = np.cos(np.radians(g)), np.sin(np.radians(g));
121        return np.array([
122            [c,0,s,0],
123            [0,1,0,0],
124            [-s,0,c,0],
125            [0,0,0,1]
126            ], dtype=np.float32)
127
```

```python
def main():
    if not glfw.init(): return
    glfw.window_hint(glfw.CONTEXT_VERSION_MAJOR, 3)
    glfw.window_hint(glfw.CONTEXT_VERSION_MINOR, 3)
    glfw.window_hint(glfw.OPENGL_PROFILE, glfw.OPENGL_CORE_PROFILE)

    janela = glfw.create_window(800, 600, "Cubo: Filtro Linear + Mipmaps", None, None)
    if not janela:
        glfw.terminate()
        return

    glfw.make_context_current(janela)
    glfw.swap_interval(1) # VSync: trava na taxa de atualização do monitor
    glEnable(GL_DEPTH_TEST) # Z-Buffer ativado (esconde a face de trás)

    shader = OpenGL.GL.shaders.compileProgram(
        OpenGL.GL.shaders.compileShader(VERTEX_SHADER, GL_VERTEX_SHADER),
        OpenGL.GL.shaders.compileShader(FRAGMENT_SHADER, GL_FRAGMENT_SHADER)
    )
```

```
    # 24 Vértices (4 por face)
    # Formato: [ X, Y, Z,   U, V ]
    vertices = np.array([
        # Face Frontal (Z = 0.5)
        -0.5, -0.5,  0.5,   0.0, 0.0, # Canto Inferior Esquerdo
         0.5, -0.5,  0.5,   1.0, 0.0, # Canto Inferior Direito
         0.5,  0.5,  0.5,   1.0, 1.0, # Canto Superior Direito
        -0.5,  0.5,  0.5,   0.0, 1.0, # Canto Superior Esquerdo

        # Face Traseira (Z = -0.5)
        -0.5, -0.5, -0.5,   1.0, 0.0,
         0.5, -0.5, -0.5,   0.0, 0.0,
         0.5,  0.5, -0.5,   0.0, 1.0,
        -0.5,  0.5, -0.5,   1.0, 1.0,

        # Face Esquerda (X = -0.5)
        -0.5, -0.5, -0.5,   0.0, 0.0,
        -0.5, -0.5,  0.5,   1.0, 0.0,
        -0.5,  0.5,  0.5,   1.0, 1.0,
        -0.5,  0.5, -0.5,   0.0, 1.0,
```

Interp

Na prática

cubo_textu

Código:

```
    # Face Direita (X = 0.5)
     0.5, -0.5, -0.5,   1.0, 0.0,
     0.5, -0.5,  0.5,   0.0, 0.0,
     0.5,  0.5,  0.5,   0.0, 1.0,
     0.5,  0.5, -0.5,   1.0, 1.0,

    # Face Superior (Y = 0.5)
    -0.5,  0.5, -0.5,   0.0, 1.0,
     0.5,  0.5, -0.5,   1.0, 1.0,
     0.5,  0.5,  0.5,   1.0, 0.0,
    -0.5,  0.5,  0.5,   0.0, 0.0,

    # Face Inferior (Y = -0.5)
    -0.5, -0.5, -0.5,   0.0, 0.0,
     0.5, -0.5, -0.5,   1.0, 0.0,
     0.5, -0.5,  0.5,   1.0, 1.0,
    -0.5, -0.5,  0.5,   0.0, 1.0,
], dtype=np.float32)
```

Interpolação de Textura

```
189     # conectando os vértices em triângulos
190     indices = np.array([
191         0, 1, 2,  2, 3, 0,        # Frontal
192         4, 5, 6,  6, 7, 4,        # Traseira
193         8, 9, 10, 10, 11, 8,      # Esquerda
194         12, 13, 14, 14, 15, 12,   # Direita
195         16, 17, 18, 18, 19, 16,   # Superior
196         20, 21, 22, 22, 23, 20    # Inferior
197     ], dtype=np.uint32)
198
```

Interpolação de Textura

```
199     # MEMÓRIA DA GPU
200     VAO = glGenVertexArrays(1)
201     VBO = glGenBuffers(1)
202     EBO = glGenBuffers(1)
203
204     glBindVertexArray(VAO)
205
206     glBindBuffer(GL_ARRAY_BUFFER, VBO)
207     glBufferData(GL_ARRAY_BUFFER, vertices.nbytes, vertices, GL_STATIC_DRAW)
208     glBindBuffer(GL_ELEMENT_ARRAY_BUFFER, EBO)
209     glBufferData(GL_ELEMENT_ARRAY_BUFFER, indices.nbytes, indices, GL_STATIC_DRAW)
210
211     # passo da memória: 5 floats por vértice (X, Y, Z, U, V)
212     passo = 5 * vertices.itemsize
```

Interpolação de Textura

Na prática

```
214     # Atributo 0: geometria (3 floats). Offset = 0.
215     glVertexAttribPointer(0, 3, GL_FLOAT, GL_FALSE, passo, ctypes.c_void_p(0))
216     glEnableVertexAttribArray(0)
217
218     # Atributo 1: Textura UV (2 floats). Offset = Pule os 3 primeiros floats (3 * bytes do float).
219     glVertexAttribPointer(1, 2, GL_FLOAT, GL_FALSE, passo, ctypes.c_void_p(3 * vertices.itemsize))
220     glEnableVertexAttribArray(1)
221
222     glBindVertexArray(0) # Fecha o VAO
223
```

Interpolação de Textura

```
224     matriz_proj = matriz_perspectiva(45.0, 800/600, 0.1, 100.0)
225     matriz_view = matriz_translacao(0, 0, -3.0)
226
227     # Lendo o arquivo do disco!
228     textura_id = carregar_textura_do_disco("C:\workspace\computacao-grafica-python\pedras.jpg")
229
230     loc_mvp = glGetUniformLocation(shader, "MVP")
231     loc_tex = glGetUniformLocation(shader, "textura_bloco")
232
233     rot_x, rot_y = 25.0, -30.0
234     velocidade = 2.0
235
236     print("Tudo pronto...")
237
```

Interpolação de Textura

```
238     # loop de renderização
239     while not glfw.window_should_close(janela):
240         glfw.poll_events()
241 
242         # entrada do teclado
243         if glfw.get_key(janela, glfw.KEY_RIGHT) == glfw.PRESS: rot_y += velocidade
244         if glfw.get_key(janela, glfw.KEY_LEFT) == glfw.PRESS:  rot_y -= velocidade
245         if glfw.get_key(janela, glfw.KEY_DOWN) == glfw.PRESS:  rot_x += velocidade
246         if glfw.get_key(janela, glfw.KEY_UP) == glfw.PRESS:    rot_x -= velocidade
247 
248         # cor de fundo
249         glClearColor(0.5, 0.8, 1.0, 1.0)
250         glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT)
251 
252         glUseProgram(shader)
253 
```

Interpolação de Textura

```
253
254         # atualiza a Matriz MVP
255         matriz_model = matriz_rotacao_x(rot_x) @ matriz_rotacao_y(rot_y)
256         MVP = matriz_proj @ matriz_view @ matriz_model
257         glUniformMatrix4fv(loc_mvp, 1, GL_FALSE, np.ascontiguousarray(MVP.T, dtype=np.float32))
258
259         # vincula a Textura no Slot 0
260         glActiveTexture(GL_TEXTURE0)
261         glBindTexture(GL_TEXTURE_2D, textura_id)
262         # Avisa o Fragment Shader para ler do Slot 0
263         glUniform1i(loc_tex, 0)
264
265         # desenhando os 36 vértices indexados (12 triângulos)
266         glBindVertexArray(VAO)
267         glDrawElements(GL_TRIANGLES, len(indices), GL_UNSIGNED_INT, None)
268
269         glfw.swap_buffers(janela)
270         # fim do while
```

Interpolação de Textura

Na prática

cubo_textura.py

Código:

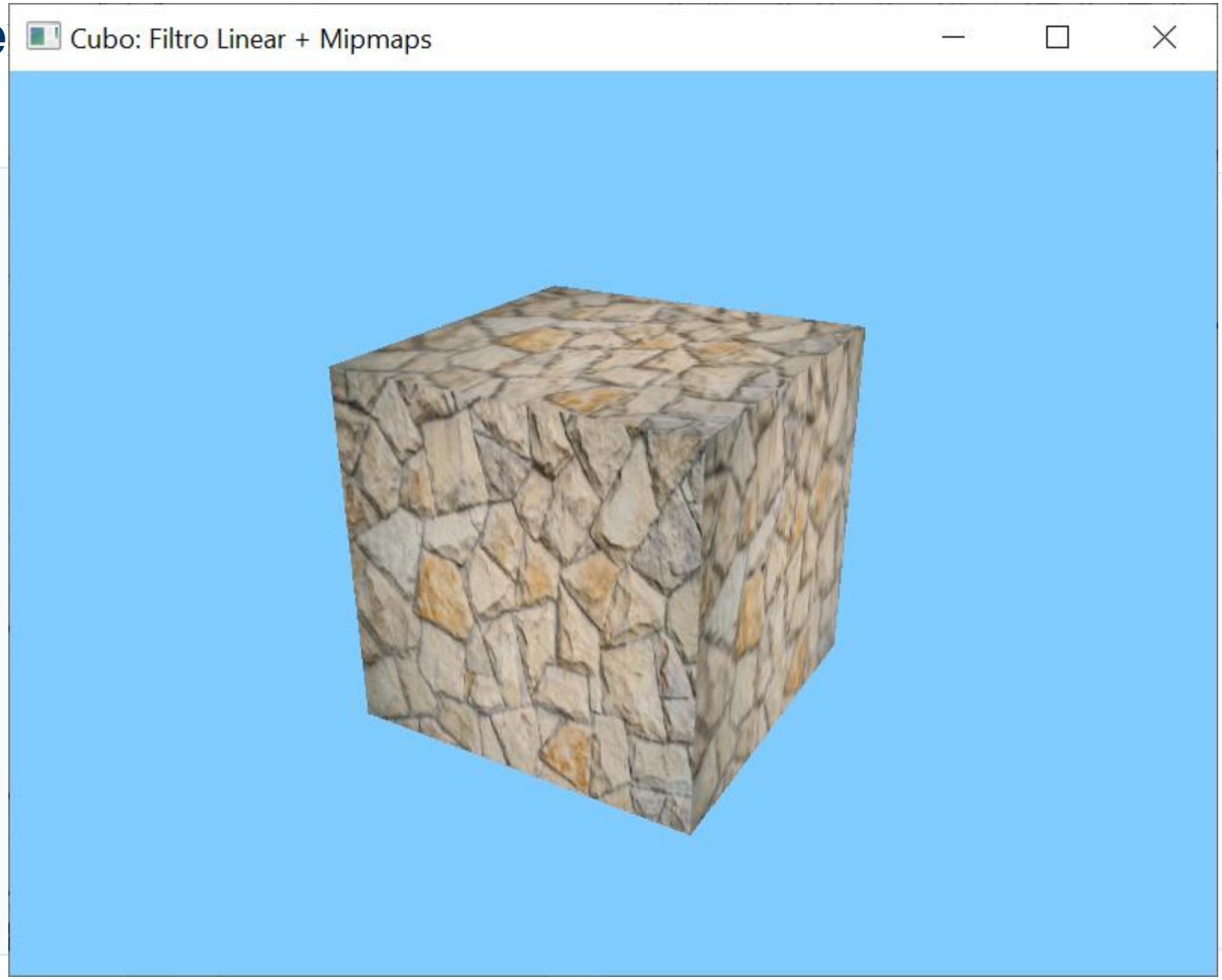
```
272         glfw.terminate()
273         # fim do main
274
275 if __name__ == "__main__":
276     main()
```

Interpolação de Te

Na prática

cubo_textura.py

Código:



Dúvidas e Discussão

Este é o momento para aprofundarmos nos conceitos.

raulivan@cefetmg.br

SIGAA

Vamos praticar

Casa 3D

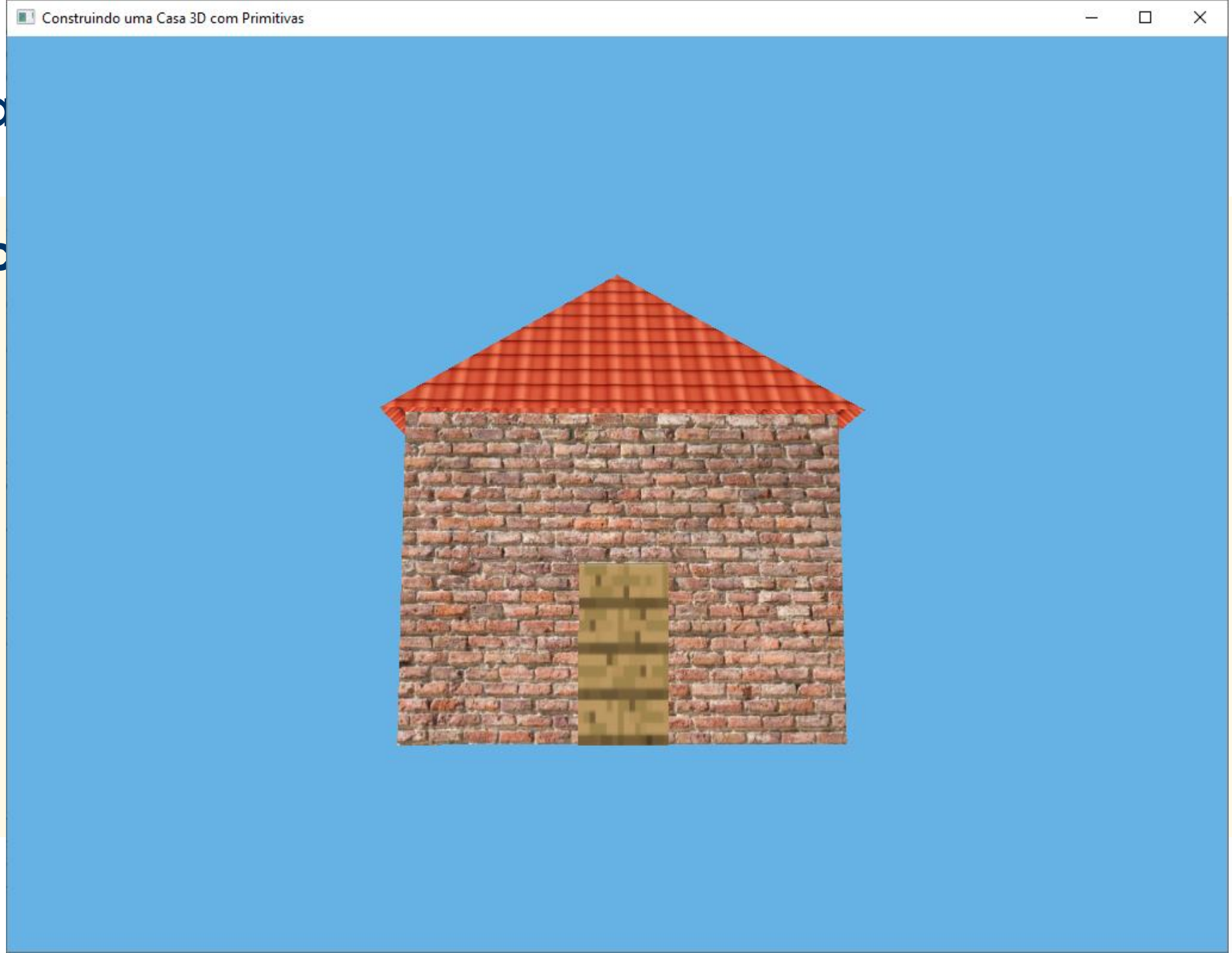
Construa uma casa 3D usando apenas

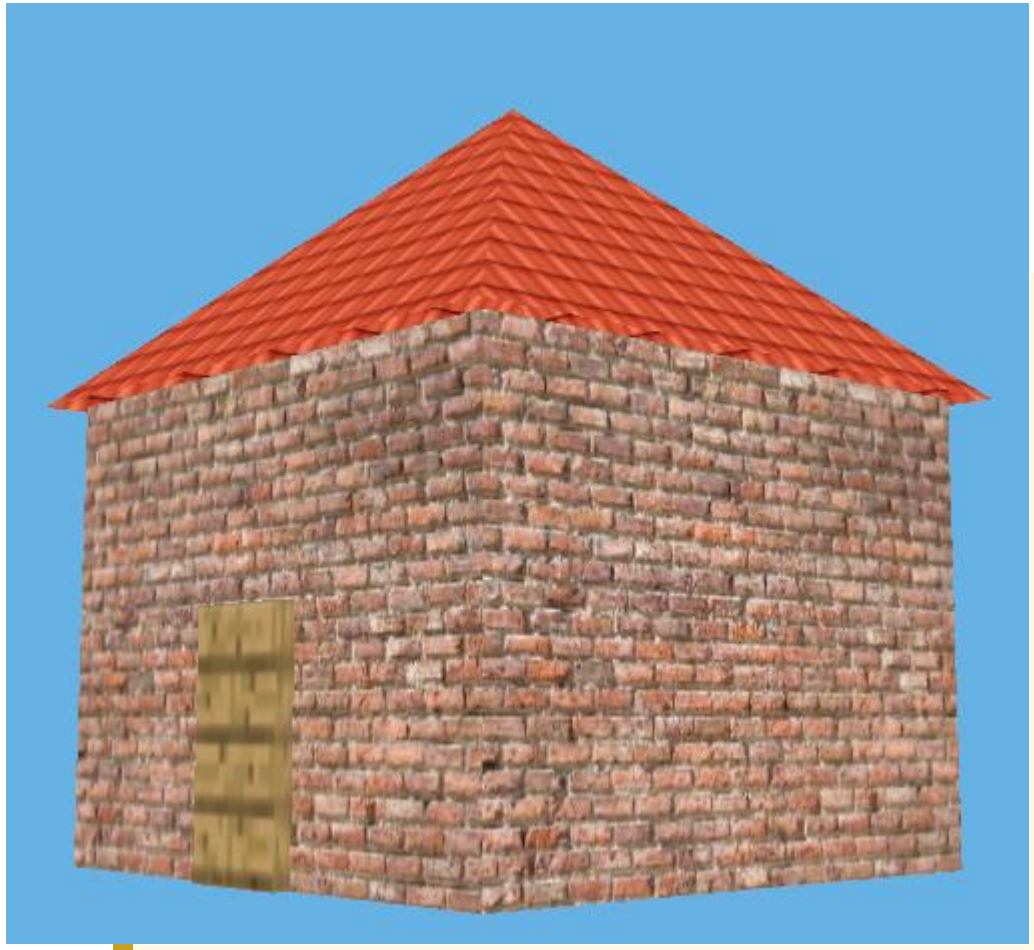
- Cubos/quadrados
- Pirâmides/triângulos
- Texturas

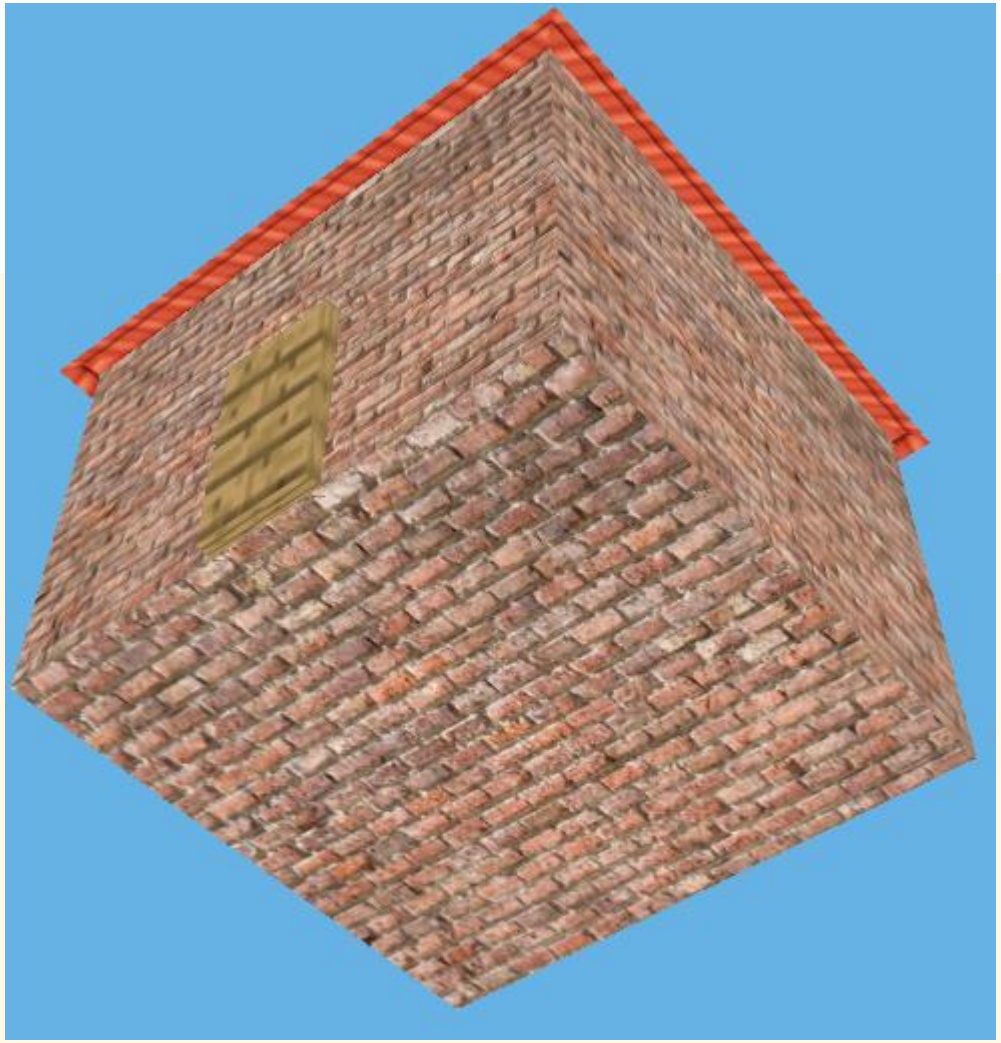
Inclua rotações usando a setas do teclado

Vo

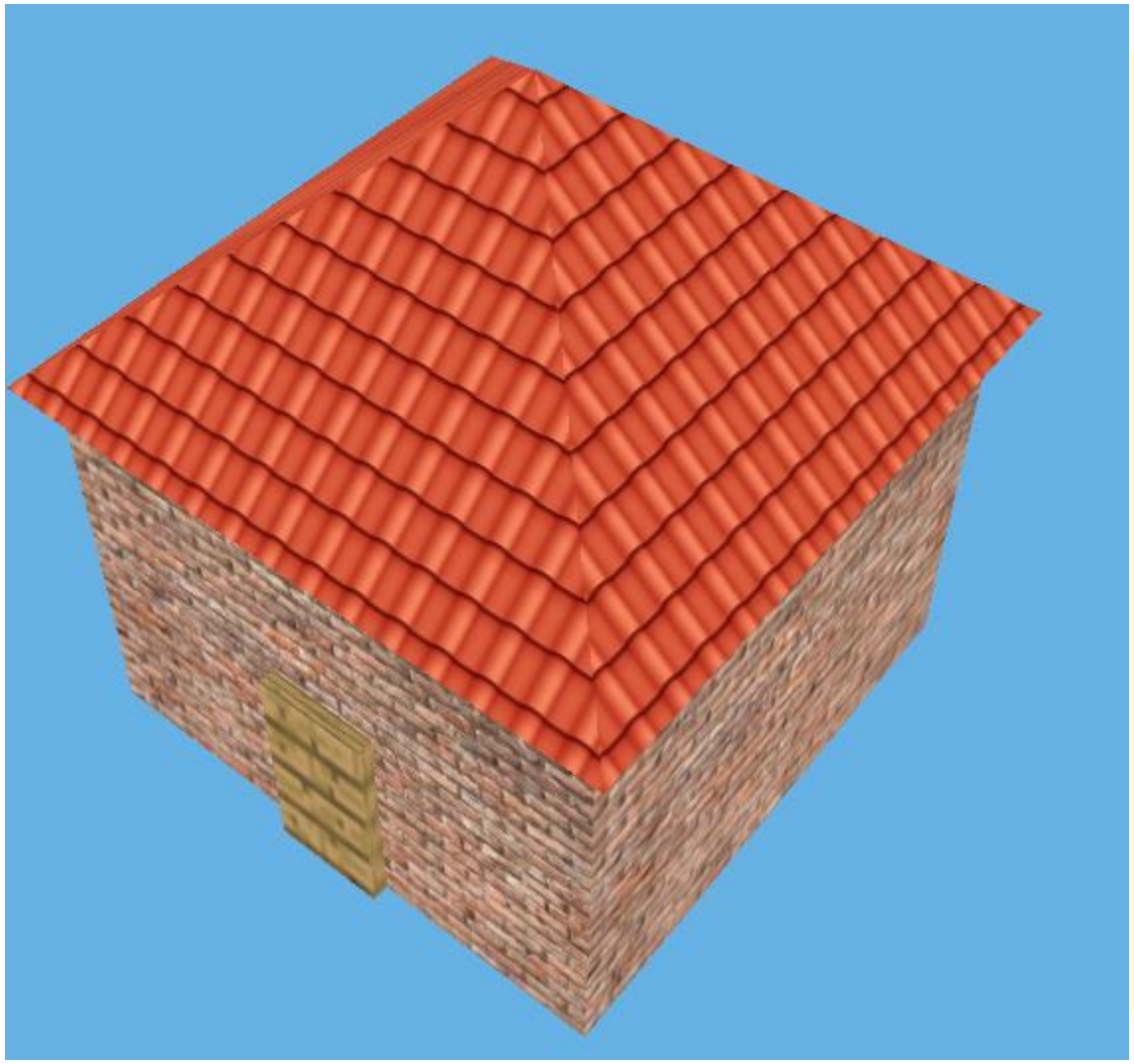
c







Vamos praticar





Próxima Aula

Projeções Geométricas...